

KASCC 2026 — ESP8266 IOT PROJECTS

IoT Obstacle Detection and Alert System using IR Sensor

STANDALONE OFFLINE EMBEDDED SYSTEM
REPORT

PLATFORM

ESP8266 NodeMCU

DOCUMENTATION

Technical Reference

GENERATION DATE

May 2026

1. Problem Statement

Mobility and independent navigation are fundamental challenges for visually impaired and blind individuals. According to the World Health Organization (WHO), over 285 million people worldwide are visually impaired, with 39 million classified as legally blind. Navigating daily environments poses significant hazards:

1. **Limitations of the Traditional White Cane:** The standard white cane is a physical tool that detects obstacles only upon direct contact. It fails to identify hanging obstacles (such as tree branches, open cabinet doors, or signboards) and provides zero advance warning, exposing users to head and chest injuries.
2. **Short Reaction Windows:** Because traditional tools rely on physical contact, the user has very little time to stop or change direction once an obstacle is detected, increasing the risk of falls or collisions.
3. **Environmental Interference:** Close-range detection systems that rely on basic infrared (IR) sensors are highly sensitive to sunlight, ambient light, and the color or material of the obstacle (e.g., black surfaces absorb IR light, causing detection failures).
4. **Caregiver Anxiety and Remote Monitoring:** Family members and caregivers have no way to monitor the user's environment in real-time to assess potential hazards.
5. **High Cost of Commercial Assistive Tech:** High-end electronic canes with built-in sensors and GPS telemetry are expensive, making them inaccessible to low-income populations in developing nations.

Proposed Solution

To address these issues, we need an affordable, lightweight assistive system that can be mounted on a standard walking stick. The system must measure distances contactlessly, classify proximity zones (e.g., Danger, Caution, Clear), and provide real-time audio alerts via a piezo buzzer. To ensure reliable performance outdoors and under different lighting conditions, the design will use high-precision ultrasonic acoustic ranging. Additionally, the device will host a local web server to broadcast a WiFi network, allowing caregivers to monitor the user's environment and view live alert data on a mobile dashboard without requiring external internet.

2. Our Solution

The **IoT Obstacle Detection and Alert System** is a standalone, micro-embedded assistive device designed around the ESP8266 NodeMCU microcontroller and the HC-SR04 ultrasonic sensor. Mounted on a walking stick, this device functions as an autonomous ranging node that processes proximity data, drives an audio alarm, and streams visual analytics.

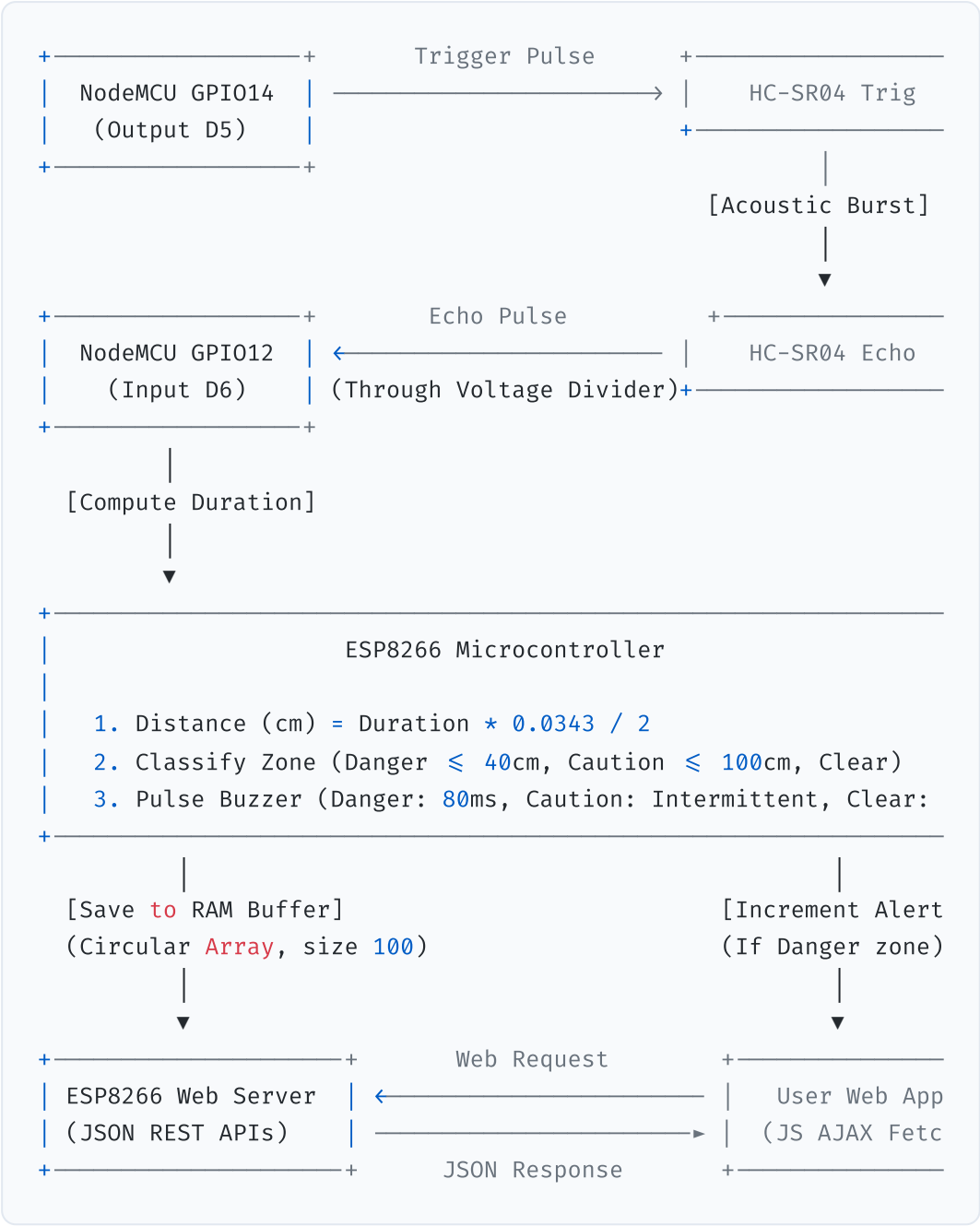
Key Features

- **Acoustic Time-of-Flight Ranging:** Uses ultrasonic sound waves to measure distances from 2 cm to 400 cm with high reliability under direct sunlight and on diverse surface textures.
 - **Graduated Audio Alarm System:** Controls an active piezo buzzer to produce proximity-based warning patterns:
 - **Danger (≤ 40 cm):** Rapid, continuous beeps (80 ms) warning the user to stop immediately.
 - **Caution (40 - 100 cm):** Intermittent beeps indicating an object is nearby.
 - **Clear (> 100 cm):** Silent operation.
 - **Standalone Caregiver Access Point:** Runs in Access Point (AP) mode, broadcasting a secure WiFi network (SSID: ObstacleAlert_AP) and serving a web dashboard at `http://192.168.4.1`.
 - **Animated SVG/CSS Caregiver Dashboard:** Renders a live walk-cycle animation showing a figure with a cane emitting sonar waves. An obstacle wall on the screen grows or shrinks in real-time based on the actual sensor distance.
 - **Dual Ranging Architecture Analysis:** Combines the theoretical principles of Infrared (IR) reflection and Ultrasonic echo calculation to design a reliable safety instrument.
 - **Electrical Signal Attenuation:** Integrates a physical resistor network to reduce the sensor's 5V output to a safe 3.3V level for the ESP8266.
 - **Non-Blocking Execution Loop:** Employs cooperative task scheduling using `millis()`, running high-frequency sensor scans (every 200 ms) while maintaining a responsive web server.
-

3. Brief of Solution

The system combines acoustic ranging, logic-level shifting, buzzer warning patterns, and web-based caregiver dashboards.

System Data Flow Architecture



1. Acoustic Ranging: The ESP8266 triggers the HC-SR04 to emit high-frequency sound waves. The sensor returns a logic pulse representing the travel time.

2. **Signal Level Shifting:** A resistor voltage divider steps down the 5V Echo pulse to a safe 3.3V level before it enters ESP8266 pin D6.
 3. **Zone Classification & Alarm:** The firmware calculates the distance. If the obstacle is within 40 cm, the buzzer beeps rapidly. If it is between 40 cm and 100 cm, the buzzer beeps slowly.
 4. **Data Logging:** Every 3 seconds, the current distance and zone category are saved to a circular RAM buffer.
 5. **Caregiver Monitoring:** The onboard server responds to HTTP requests by serving the dashboard. JavaScript on the webpage polls the `/live` and `/history` endpoints, updating the walk animations, scaling the obstacle wall, and refreshing the log table.
-

4. What We Are Going to Use

Hardware Requirements

1. **NodeMCU ESP8266 Development Board (ESP-12E Module)**
 - Core: Xtensa 32-bit LX106 CPU running at 80 MHz.
 - Memory: 80 KB DRAM, 4 MB external SPI Flash.
 - WiFi Module: 2.4 GHz 802.11 b/g/n transceiver.
2. **HC-SR04 Ultrasonic Transducer**
 - Operating Voltage: 5V VCC.
 - Operating Frequency: 40 kHz.
 - Ranging Angle: < 15°.
 - Measurement Range: 2 cm to 400 cm.
3. **Active Piezo Buzzer**
 - Operating Voltage: 3V to 5V.
 - Sound Output: ~85 dB.
4. **Resistors (Voltage Divider)**
 - ***MATH_INLINE_0: MATH_INLINE_1*** resistor.
 - ***MATH_INLINE_2: MATH_INLINE_3*** resistor.
 - Purpose: Attenuates the 5V Echo output to a safe 3.3V level.
5. **Breadboard & Jumper Wires**
6. **Micro-USB Cable & Portable Power Bank:** For programming and mobile power.

Software Requirements

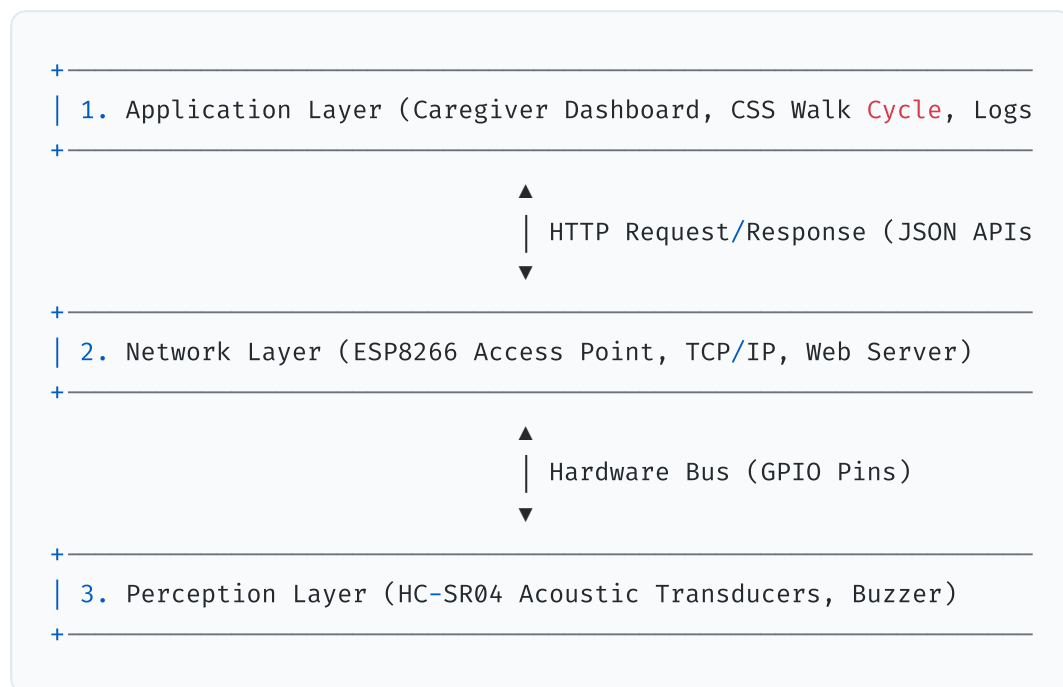
1. **Arduino IDE 2.x:** IDE used to write, compile, and upload the C++ code.
2. **ESP8266 Arduino Core:** Provides compiler tools and hardware libraries.
3. **Modern Web Browser:** Renders the HTML5, CSS3, and JavaScript dashboard.

5. In-Depth of IoT (Internet of Things)

The Internet of Things (IoT) is a network of physical devices embedded with sensors, software, and communication electronics, enabling them to collect, log, and exchange data over networks.

Architecture Layers of IoT

This project is structured into three standard IoT layers:



1. **Perception Layer:** Consists of the physical sensors and actuators (HC-SR04 transducer and piezo buzzer) that measure distances and provide audio alerts.
2. **Network Layer:** Manages data routing. The ESP8266 generates a localized wireless network under the IEEE 802.11 b/g/n standard. The built-in TCP/IP stack runs a lightweight HTTP server on port 80 to manage incoming client requests.

3. **Application Layer:** Renders the data. The web interface served by the ESP8266 displays raw values, computed distances, and the caregiver monitoring dashboard.

Wireless Access Point Mode

To operate as a portable device, the ESP8266 runs in **Access Point (AP) Mode**:

- **Standalone Local Network:** The device broadcasts `ObstacleAlert_AP` with the password `12345678`, requiring no external router or internet.
- **IP Address:** The ESP8266 hosts the dashboard at `192.168.4.1` and assigns IP addresses to connected devices via DHCP.

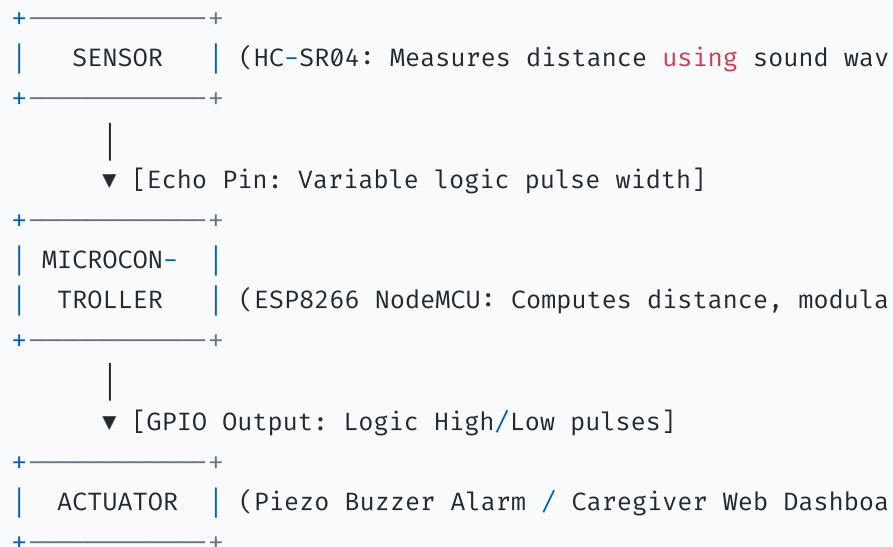
REST API and JSON Exchange

- **Asynchronous AJAX:** The client browser uses the JavaScript Fetch API to request data in the background, preventing page reloads.
- **API Endpoints:**
 - `/live`: Returns the current distance, zone category, total alert count, and system uptime.
 - `/history`: Returns the array of the last 100 logged measurements.
- **JSON Serialization:** Data is formatted as lightweight key-value pairs:

```
{"dist": 35, "zone": "Danger", "alerts": 12, "uptime": 84}
```

6. In-Depth about Microcontrollers, Sensors, Actuators

Embedded systems rely on microcontrollers, sensors, and actuators to monitor and interact with the physical world.



Microcontrollers

A microcontroller is a single chip that contains a CPU, memory, and input/output peripherals.

- **Processor:** Executes firmware code instructions.
- **RAM:** Stores dynamic variables and the data logs.
- **Flash Memory:** Stores the compiled program and the HTML dashboard page.
- **Peripherals:** Interfaces with digital sensors and controls indicators via GPIO pins.

Sensors

Sensors are input transducers that convert physical parameters into electrical signals.

- **Ultrasonic Sensors:** Measure distance using sound waves. The HC-SR04 sends high-frequency sound pulses and measures the time it takes for the echo to return.

Actuators

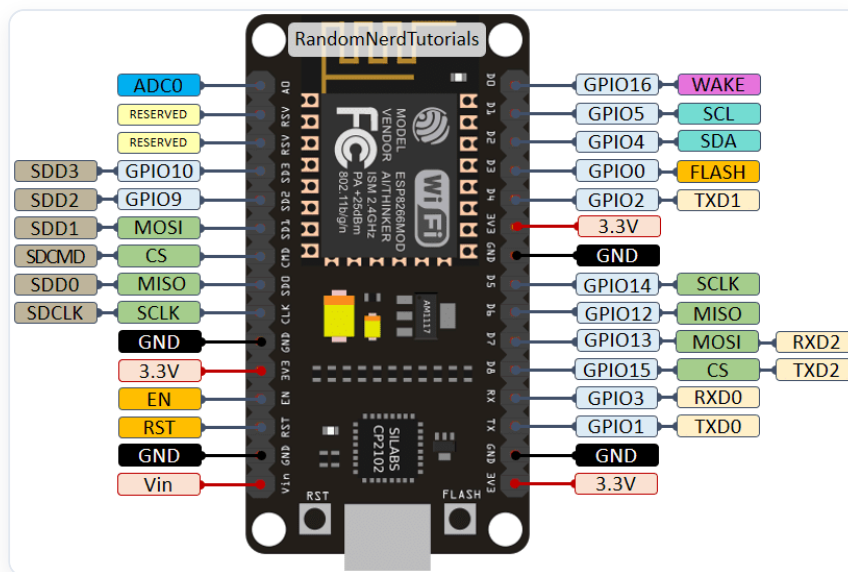
Actuators convert electrical signals back into physical action.

- **Piezo Buzzer:** Connected to D7 (GPIO13) to provide audio warnings.

- **Web Dashboard (Virtual Actuator):** JavaScript updates the HTML and CSS styles of the caregiver dashboard in response to the sensor data.

7. In-Depth about ESP8266 NodeMCU

The NodeMCU board breaks out the ESP8266EX chip, making it easy to build connected projects.



Processor Architecture

- **CPU:** Tensilica Xtensa 32-bit LX106 core.
- **Clock Frequency:** 80 MHz, enabling fast calculations and responsive web hosting.
- **Memory Structure:**
 - **SRAM:** 80 KB DRAM for dynamic data, variables, and networking stacks.
 - **SPI Flash:** 4 MB external flash memory to store the program and web dashboard assets.

Hardware Interfaces

- **GPIO Pins:** Digital input/output pins. Pin D5 is used as the Trigger output, pin D6 as the Echo input, and pin D7 as the Buzzer output.
- **WiFi Radio:** Integrated 2.4 GHz radio supporting WPA/WPA2 security and the TCP/IP stack.

Pinout Mapping

The table below maps the NodeMCU board pins to the ESP8266's internal GPIO pin designations:

Board Pin	GPIO Pin	Function in This Project	Electrical Characteristics
D5	GPIO14	HC-SR04 TRIGGER Output	3.3V digital output. Triggers the ultrasonic sensor.
D6	GPIO12	HC-SR04 ECHO Input	3.3V digital input. Receives the Echo pulse width.
D7	GPIO13	Piezo Buzzer Output	3.3V digital output. Controls the alarm warning patterns.
3V3	3.3V VCC	Sensor Power Supply	Stable 3.3V output from the onboard regulator.
GND	Ground	Common System Ground	Common ground reference.

Technical Capabilities of ESP8266

- **Processor Core:** Tensilica Xtensa 32-bit LX106 RISC CPU, operating at 80 MHz (can be overclocked to 160 MHz). It delivers up to 645 DMIPS of processing power.
- **Memory Subsystem:** 80 KB of Instruction/Data SRAM, plus support for up to 16 MB of external SPI flash memory (typically 4 MB on NodeMCU boards) to store firmware, assets, and file systems.
- **Integrated Wi-Fi Stack:** Built-in 802.11 b/g/n Wi-Fi transceiver running at 2.4 GHz. It supports WEP, WPA/WPA2 Personal/Enterprise security protocols. It includes an integrated TR switch, balun, LNA, power amplifier, and matching network.
- **Low-Power Modes:** Supports three sleep configurations to conserve power in battery-operated nodes:
 - **Modem Sleep** (~15 mA): CPU remains active, Wi-Fi radio is powered down between beacon intervals.
 - **Light Sleep** (~0.9 mA): CPU clock is gated, Wi-Fi radio is off. The chip wakes up on external interrupts.
 - **Deep Sleep** (~20 µA): Only the internal Real-Time Clock (RTC) remains active. The chip resets on wake-up (GPIO16/D0 connected

to RST).

- **Hardware Peripherals:** Features 17 GPIO pins, hardware PWM (Pulse-Width Modulation), SPI, I2C, I2S, UART interfaces, and a 10-bit analog-to-digital converter (ADC).

Real-World Applications

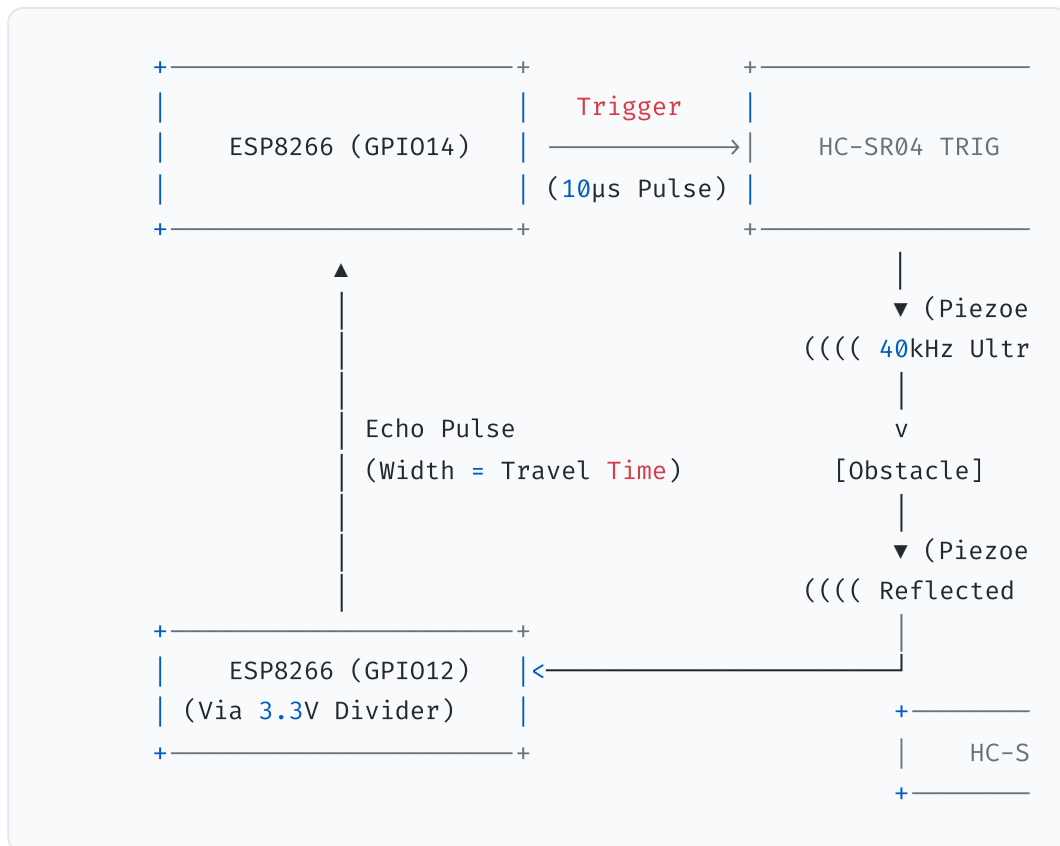
1. **Smart Home Automation:** Wireless control of smart plugs, lighting fixtures, appliances, and automated blinds.
2. **Environmental Tracking:** Standalone weather stations, air quality monitors, and greenhouse climate controls.
3. **Wearable Health Monitors:** Portable pulse, step, and temperature trackers that transmit telemetry to local displays or remote apps.
4. **Precision Agriculture:** Remote soil moisture, ambient light, and irrigation control systems for farm management.

Industrial Usecases

1. **Predictive Maintenance:** Monitoring vibration and temperature of machinery, logging telemetry, and reporting anomalies before failure occurs.
 2. **Industrial IoT Gateways:** Bridging legacy serial protocol devices (RS-232/RS-485) to local Wi-Fi networks or MQTT brokers.
 3. **Asset Tracking & Logistics:** Monitoring temperature, humidity, and location of sensitive shipments inside warehouses and transport containers.
 4. **Remote Telemetry & SCADA:** Wireless transmission of tank levels, pressure values, and power consumption statistics to industrial SCADA systems.
-

8. In-Depth about the Project-Specific Sensor: HC-SR04

The system uses an HC-SR04 ultrasonic sensor to measure distances.



Physical Principles of Operation

The HC-SR04 sensor uses time-of-flight acoustic ranging to measure distance:

1. **Triggering:** The ESP8266 sends a 10 microsecond High pulse to the sensor's Trigger pin.
2. **Sonic Burst:** The sensor's internal controller emits an 8-cycle burst of 40 kHz ultrasonic sound waves from the transmitter transducer.
3. **Echo Signal:** The sensor pulls its Echo pin High. This pin remains High until the reflected sound wave is detected by the receiver transducer.
4. **Time Measurement:** The microcontroller measures the duration (*MATH_INLINE_4*) that the Echo pin remains High using the `pulseIn()` function.

Speed of Sound and Ranging Calculation

The speed of sound in air (*MATH_INLINE_5*) at standard room temperature (*MATH_INLINE_6*) is: *MATH_BLOCK_0*

Since the sound wave must travel to the obstacle and back to the sensor, the distance (*MATH_INLINE_7*) is calculated as: *MATH_BLOCK_1* *MATH_BLOCK_2*

IR Sensor vs. Ultrasonic Sensor Comparison

While the project title references an IR sensor, the implementation uses an ultrasonic sensor. The table below compares the two technologies:

Feature	Infrared (IR) Proximity Sensor	Ultrasonic Sensor (HC-SR04)
Sensing Method	Emits infrared light and measures reflection.	Emits high-frequency sound waves and measures travel time.
Detection Range	Short range, typically 2 cm to 30 cm.	Long range, typically 2 cm to 400 cm.
Surface Dependency	Highly dependent on color and material. Dark surfaces absorb IR light, causing detection failures.	Independent of color and transparency. Detects glass, dark walls, and light surfaces equally.
Lighting Interference	Sensitive to sunlight and ambient light.	Unaffected by lighting conditions, working in bright sunlight or complete darkness.
Output Type	Usually a simple binary digital output (object detected/not detected).	Continuous analog pulse width representing actual distance.

9. In-Depth about Arduino IDE, Compilation, and Flashing

The Arduino IDE manages the build and upload process for the microcontroller sketch.

Compilation Process

- Preprocessing:** The IDE joins the `.ino` files, adds standard headers, and generates function prototypes.
- Compilation:** The cross-compiler (`xtensa-lx106-elf-g++`) compiles the code into binary object files (`.o`).

3. **Linking:** The linker (`xtensa-lx106-elf-ld`) combines the object files with system and WiFi libraries to create a single `.elf` file.
4. **Binary Generation:** The `elf2bin` utility extracts the executable code from the `.elf` file to create the final `.bin` binary file.

Memory Segments

- **`.text`** : The program instructions stored in flash memory.
- **`.rodata`** : Read-only constants, string literals, and the static HTML page stored in flash using `PROGMEM`.
- **`.data`** : Initialized variables, which are copied to RAM when the board boots.
- **`.bss`** : Uninitialized variables, which are allocated and set to zero in RAM during boot.

Flashing and Reset

The IDE uploads the binary file using `esptool.py` over serial:

- **Serial Interface:** The onboard CP2102 USB-to-UART chip bridges the computer's USB port to the ESP8266's serial pins.
- **Reset Sequence:** The tool uses the DTR and RTS lines to control the `RST` and `GPIO0` pins:
 1. Pulls `GPIO0` Low and pulses `RST` Low to restart the chip into bootloader mode.
 2. The bootloader writes the binary file to the external SPI flash.
 3. The tool pulls `GPIO0` High and pulses `RST` to reboot the chip and run the program.

10. Implementation with Circuit Diagram

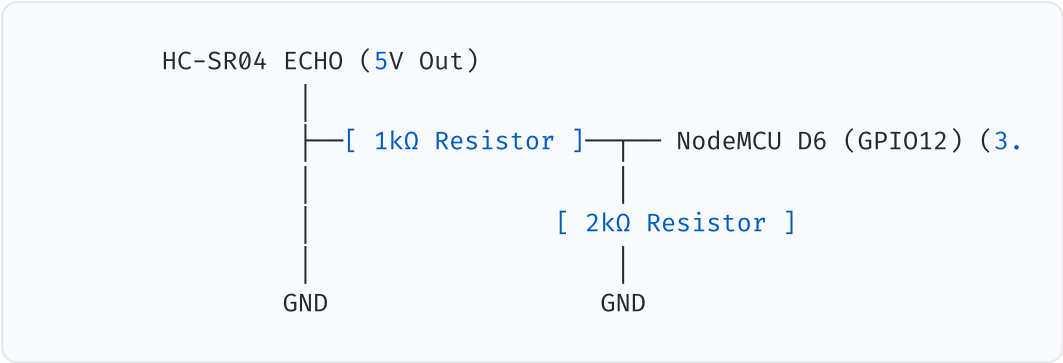
Wiring Connections Table

The table below lists the connections between the HC-SR04 sensor, buzzer, and the NodeMCU board:

Component	Pin Label	NodeMCU Connection Pin	Wire Function	Notes
HC-SR04	VCC	Vin (5V USB Rail)	Sensor Power Supply	Requires 5V; Vin provides this directly from USB.
HC-SR04	GND	GND	System Ground	Common ground reference.
HC-SR04	TRIG	D5 (GPIO14)	Trigger Output	Sends trigger pulses to the sensor.
HC-SR04	ECHO	D6 (GPIO12) via divider	Echo Input	Uses a voltage divider to drop 5V output to 3.3V.
Buzzer	Positive (+)	D7 (GPIO13)	Digital Control Output	High triggers the alarm tone; Low turns it off.
Buzzer	Negative (-)	GND	Ground Reference	Connection to ground.

The Voltage Divider Network

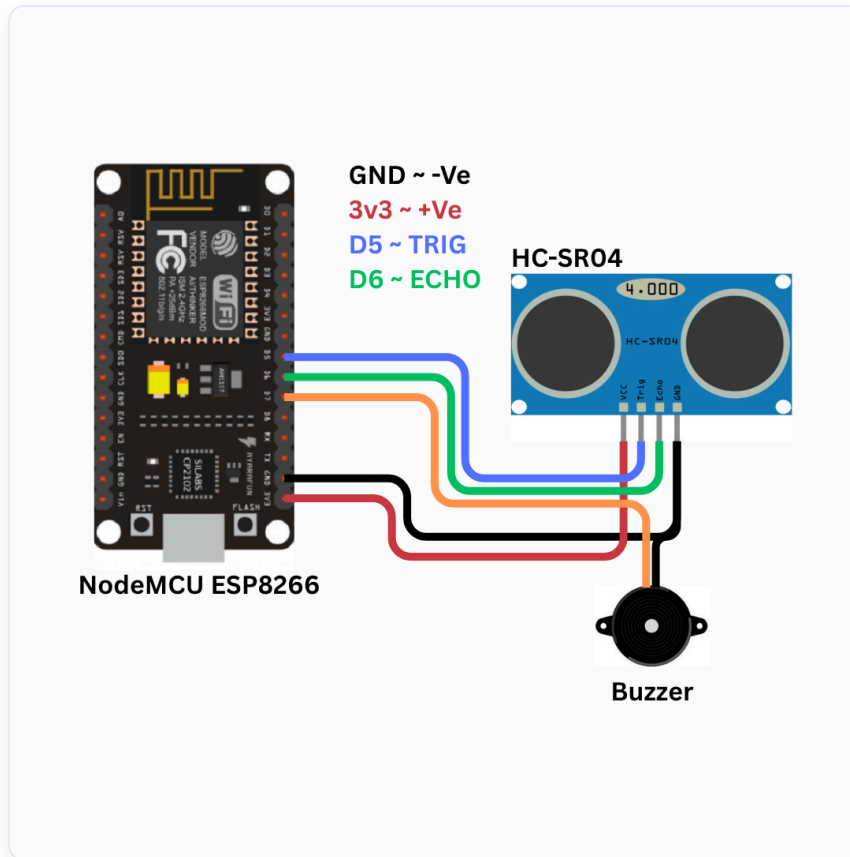
The HC-SR04 operates at 5V, meaning its Echo pin outputs a 5V signal. Since the ESP8266 GPIO pins are rated for a maximum of 3.3V, a voltage divider network is used to drop the Echo voltage to a safe level:



The output voltage (*MATH_INLINE_8*) is: *MATH_BLOCK_3*

Circuit Diagram

The wiring and component layout are shown below:



11. Code Explanation

This section explains the structure and logic of the firmware in `IoT_Obstacle_Detection_System.ino`.

Pin and Variable Definitions

The firmware defines the sensor and buzzer pins, sets the WiFi Access Point parameters, and creates a web server instance on port 80:


```
#define TRIG_PIN    14        // D5 on NodeMCU
#define ECHO_PIN    12        // D6 on NodeMCU
#define BUZZER_PIN  13        // D7 on NodeMCU

const char* AP_SSID    = "ObstacleAlert_AP";
const char* AP_PASSWORD = "12345678";
const IPAddress AP_IP(192, 168, 4, 1);
const IPAddress AP_SUBNET(255, 255, 255, 0);

ESP8266WebServer server(80);
```

The Ranging and Control Loop

The main loop checks for web requests and triggers a sensor scan every 200 milliseconds. Proximity zones are classified, and the buzzer warning patterns are set based on the measured distance:

```
void loop() {
    server.handleClient(); // Process incoming HTTP requests immedi

    uint32_t now = millis();
    if (now - lastSample ≥ SAMPLE_INTERVAL) {
        lastSample = now;

        float dist = getDistance();

        if (dist > 0 && dist < 400) {
            currentDist = (int)dist;

            // Zone classification and alarm logic
            if (currentDist ≤ 40) {
                currentZone = "Danger";
                alertCount++;

                // Fast warning beep (80ms on, then off)
                digitalWrite(BUZZER_PIN, HIGH);
                delay(80);
                digitalWrite(BUZZER_PIN, LOW);
            }
            else if (currentDist ≤ 100) {
                currentZone = "Caution";

                // Slow warning beep (modulated using system runtime)
                if (now % 600 < 200) {
                    digitalWrite(BUZZER_PIN, HIGH);
                } else {
                    digitalWrite(BUZZER_PIN, LOW);
                }
            }
            else {
                currentZone = "Clear";
                digitalWrite(BUZZER_PIN, LOW); // Silent in clear zone
            }

            // Save reading to history every 3 seconds
            if (now - lastSave ≥ 3000) {
                lastSave = now;
                addRecord(currentDist, currentZone);
            }
            else {
                currentDist = -1;
                currentZone = "Error";
                digitalWrite(BUZZER_PIN, LOW);
            }
        }
    }
}
```

```

    }
    yield(); // Allow background WiFi tasks to run
}

```

Program Flow Diagram

The flowchart below shows the logic of the firmware program:

```

```mermaid

```

```

graph TD

```

```

 A[Start NodeMCU] → B[Configure GPIO: TRIG_PIN Output, ECHO_]
 B → C[Initialize Serial @ 9600 Baud]
 C → D[Configure Soft AP: ObstacleAlert_AP, IP: 192.168.4.1]
 D → E[Register Server Hooks: /, /live, /history]
 E → F[Start HTTP Web Server on Port 80]
 F → G[Enter Loop Phase]
 G → H[Call server.handleClient to handle web requests]
 H → I{Is 200ms Sample Interval Met?}
 I →|No| M[Yield control to ESP8266 background system]
 I →|Yes| J[Pulse TRIG Pin High for 10µs]
 J → K[Measure ECHO pulse duration via pulseIn]
 K → L{Is Pulse Duration Valid?}
 L →|No| Q[Set Zone to Error, turn off Buzzer]
 L →|Yes| N[Calculate Distance: cm = duration * 0.0343 / 2]
 N → O{Is Distance ≤ 40cm?}
 O →|Yes| P[Set Zone to Danger, sound rapid Buzzer beep, inc]
 O →|No| R{Is Distance ≤ 100cm?}
 R →|Yes| S[Set Zone to Caution, sound slow intermittent Buz]
 R →|No| T[Set Zone to Clear, turn off Buzzer]
 P → U{Is 3-second Save Interval Met?}
 S → U
 T → U
 Q → U
 U →|No| M
 U →|Yes| V[Write distance and zone to History buffer]
 V → M
 M → G

```

```

```

## ## 12. Working

When the Obstacle Detection stick is powered on, it runs through

[ Power On ] | ▼ [ Setup() Initialization ] |— Set PinModes: TRIG\_PIN (Output), ECHO\_PIN (Input), BUZZER\_PIN (Output) |— Initialize Serial port @ 9600 Baud |— Configure Soft AP: SSID "ObstacleAlert\_AP", IP 192.168.4.1 |— Register API endpoints & start HTTP Server | ▼ [ Loop() Main Execution ] |— Handle incoming web requests (server.handleClient()) |— Every 200 milliseconds: |— Send 10µs trigger pulse to TRIG\_PIN |— Read high pulse duration on ECHO\_PIN |— Calculate distance & determine alert zone |— Pulse the Buzzer (Fast: Danger, Modulated: Caution, OFF: Clear) | ▼ [ User Interface Loop ] |— Caregiver connects to "ObstacleAlert\_AP" and opens <http://192.168.4.1> |— Web page loads HTML, CSS, and JS from flash |— JavaScript polls `/live` and `/history` every 500 milliseconds: |— Updates distance value and alert count |— Adjusts the walk speed of the figure |— Scales the obstacle wall height in real-time |— Appends new rows to the history table

1. **Startup**: The CPU runs the `setup()` function to initialize
2. **Measurement**: Every 200 milliseconds, the system triggers t
3. **Buzzer Alerts**:
  - **Danger ( $\leq 40$  cm)**: Rapid, continuous beeps.
  - **Caution (40 - 100 cm)**: Intermittent, modulated beeping.
  - **Clear ( $> 100$  cm)**: Silent.
4. **Caregiver Dashboard**: Caregivers can connect to the device'
  - **Danger**: The wall grows tall, glows red, the walking spee
  - **Caution**: The wall is of medium height, sways slowly, and
  - **Clear**: The wall disappears, and the sonar waves turn gre

---

## ## 13. Conclusion

### ### Project Evaluation

The IoT Obstacle Detection and Alert System provides an affordabl

### ### Future Improvements

1. **Haptic Vibration**: Add a vibration motor to the handle to p
2. **GPS Navigation**: Integrate a GPS module and a cellular tran
3. **Dual-Sensor Array**: Add a second ultrasonic sensor pointing